

Avaliação Técnica - Desenvolvedor Front End

1. TypeScript e Qualidade de Código

1.1. Refatoração

Melhorias aplicadas:

- Tipagem forte com interfaces.
- Uso de *Constructor Assignment*.
- Programação declarativa com métodos de array.
- Imutabilidade de referências com `readonly`.
- Interpolação de strings com *Template Literals*.

```
interface IProduct {
  id: number;
  description: string;
  stockQuantity: number;
}

class Product implements IProduct {
  constructor(
    public id: number,
    public description: string,
    public stockQuantity: number
  ) {}
}

class Inventory {
  private readonly products: IProduct[] = [
    new Product(1, 'Maçã', 20),
    new Product(2, 'Laranja', 0),
    new Product(3, 'Limão', 20)
  ];

  getProductDescription(productId: number): string {
    const product = this.products.find(p => p.id === productId);
    if (!product) return 'Produto não encontrado';

    return `${product.id} - ${product.description} (${product.stockQuantity}x)`;
  }

  hasStock(productId: number): boolean {
    const product = this.products.find(p => p.id === productId);
    return !!product && product.stockQuantity > 0;
  }
}
```

1.2. Generics e tipos utilitários

```
interface PageParams {
  page: number;
  size: number;
}

interface Page<T> {
  items: T[];
  total: number;
}

function filterAndPaginate<T>(
  data: T[],
  filterFn: (item: T) => boolean,
  params: PageParams
): Page<T> {
  const filteredData = data.filter(filterFn);
  const start = (params.page - 1) * params.size;
  const end = start + params.size;

  return {
    items: filteredData.slice(start, end),
    total: filteredData.length
  };
}

interface User {
  name: string;
  email: string;
}

const users: User[] = [
  { name: 'Angelo', email: 'angelo@email.com' },
  { name: 'João', email: 'joao@email.com' },
  { name: 'Ana', email: 'ana@email.com' }
];

const result = filterAndPaginate(
  users,
  (u) => u.name.toLowerCase().includes('a'),
  { page: 1, size: 2 }
);
```

2. Angular — Fundamentos e Reatividade

2.1. Change Detection e OnPush

Identificação do Problema:

O componente utiliza a estratégia OnPush. Nesse modo, o Angular não dispara a detecção de mudanças automaticamente para eventos assíncronos que não alteram referências de `@Input` ou que não venham de eventos disparados no template.

Sugestão de Correção:

Injetar o `ChangeDetectorRef` e invocar o método `markForCheck()` no retorno da operação assíncrona.

```
constructor(  
  private readonly pessoaService: PessoaService,  
  private readonly cdr: ChangeDetectorRef  
) {}  
  
ngOnInit(): void {  
  this.pessoaService.buscarPorId(1).subscribe((pessoa) => {  
    this.texto = `Nome: ${pessoa.nome}`;  
    this.cdr.markForCheck();  
  });  
}
```

2.2. RxJS — eliminando subscriptions aninhadas

Refatoração:

Substituição do encadeamento manual pelo operador `switchMap` para garantir o cancelamento de fluxos anteriores e evitar *race conditions*. Uso do `DestroyRef` para gerenciar o ciclo de vida da inscrição de forma segura dentro do `ngOnInit`.

```
private destroyRef = inject(DestroyRef);  
  
ngOnInit(): void {  
  const pessoaId = 1;  
  
  this.pessoaService.buscarPorId(pessoaId).pipe(  
    switchMap(pessoa =>  
      this.pessoaService.buscarQuantidadeFamiliares(pessoaId).pipe(  
        map(qtd => ({ nome: pessoa.nome, qtd })))  
    ),  
    takeUntilDestroyed(this.destroyRef)  
  ).subscribe(({ nome, qtd }) => {  
    this.texto = `Nome: ${nome} | familiares: ${qtd}`;  
  });  
}
```

2.4. Performance — OnPush e trackBy

- **trackBy:** Melhora a performance ao permitir que o Angular identifique itens da lista por uma chave única. Isso evita destruições e reconstruções desnecessárias de elementos do DOM ao atualizar a lista.
- **OnPush:** Otimiza o ciclo de vida da aplicação ao limitar a detecção de mudanças. O componente só é verificado se houver mudança em seus inputs ou disparo explícito, reduzindo o custo de processamento.
- **Impacto Default:** No modo padrão, qualquer evento simples aciona a verificação em toda a árvore de componentes, o que pode causar lentidão severa em aplicações com listas extensas.

3. Gerenciamento de Estado

3.1. Angular Signals — estado local

```
import { Component, signal, computed, output, effect } from '@angular/core';

@Component({ ... })
export class CartComponent {
  items = signal<{ name: string; price: number; quantity: number }[]>([]);

  total = computed(() =>
    this.items().reduce((acc, item) => acc + (item.price * item.quantity), 0)
  );

  totalChanged = output<number>();

  constructor() {
    effect(() => this.totalChanged.emit(this.total()));
  }

  addItem(name: string, price: number) {
    this.items.update(list => [...list, { name, price, quantity: 1 }]);
  }

  removeItem(name: string) {
    this.items.update(list => list.filter(i => i.name !== name));
  }
}
```